

CSE 207: Data Structures and Algorithms-II (Part 1)

Instructor
Dr Md Monirul Islam

Text Books:

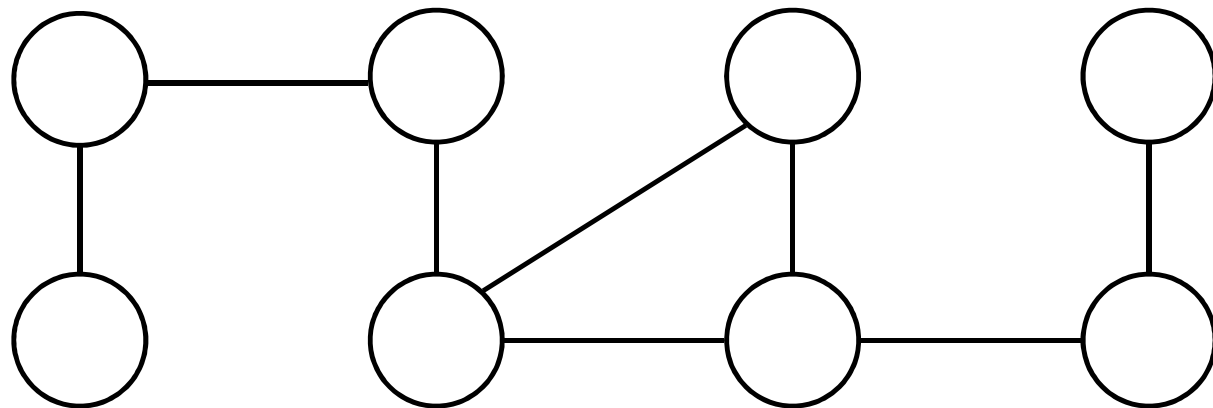
- Introduction to Algorithms
 - Thomas H. Cormen and others
- Algorithm Design
 - Jon Kleinberg and Eva Tardos

Contents in Part 1:

- Graph Algorithms:
 - minimum spanning tree algorithms
 - shortest path algorithms
 - maximum flow and maximum bipartite matching;
- Advanced data Structures
 - red-black trees
 - AVL trees
 - splay trees
 - skip list

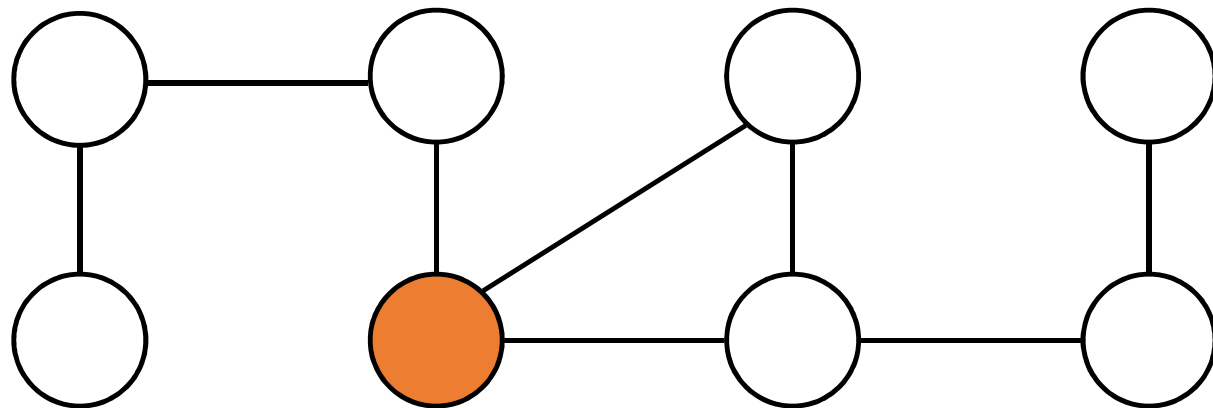
Review of Graph Search Algorithms

- “Explore” a graph, turning it into a tree
 - One vertex at a time
 - Expand frontier of explored vertices across the breadth of the frontier
- Builds a tree over the graph
 - Pick a *source vertex* to be the root
 - Find (“discover”) its children, then their children, etc.



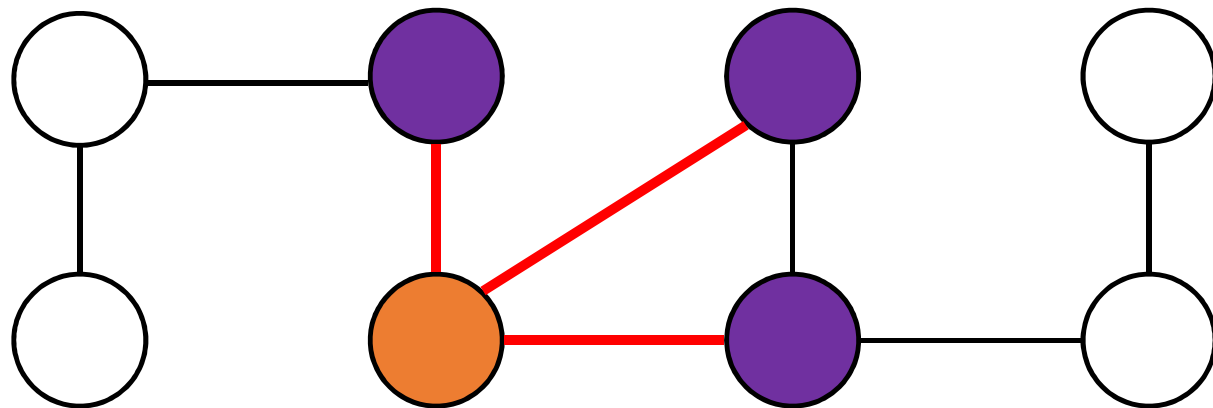
Breadth-First Search

- “Explore” a graph, turning it into a tree
 - One vertex at a time
 - Expand *frontier* of explored vertices *across* the *breadth* of the frontier
- Builds a tree over the graph
 - Pick a *source vertex* to be the root
 - Find (“discover”) its *children*, then *their children*, etc.



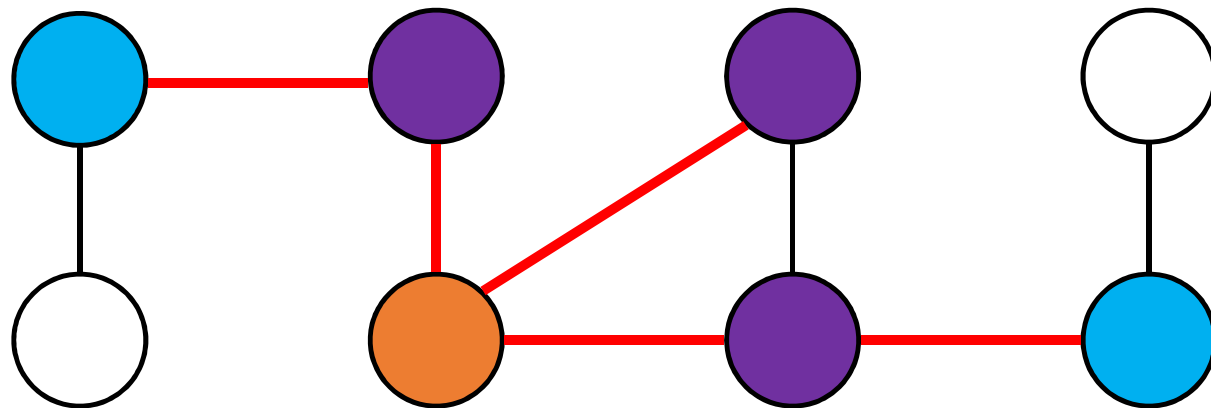
Review of Graph Search Algorithms

- “Explore” a graph, turning it into a tree
 - One vertex at a time
 - Expand *frontier* of explored vertices *across* the *breadth* of the frontier
- Builds a *tree* over the graph
 - Pick a *source vertex* to be the root
 - Find (“discover”) its *children*, then *their children*, etc.



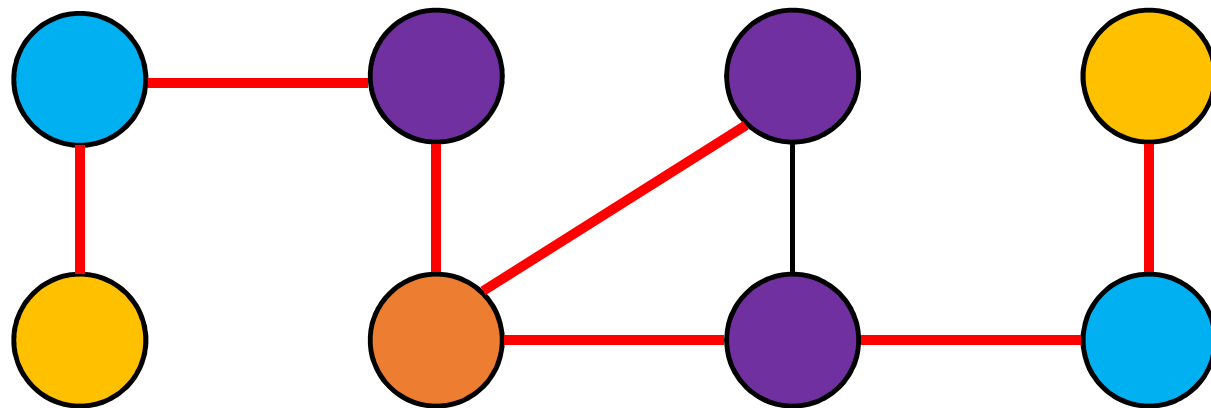
Review of Graph Search Algorithms

- “Explore” a graph, turning it into a tree
 - One vertex at a time
 - Expand *frontier* of explored vertices *across* the *breadth* of the frontier
- Builds a *tree* over the graph
 - Pick a *source vertex* to be the root
 - Find (“discover”) its *children*, then *their children*, etc.



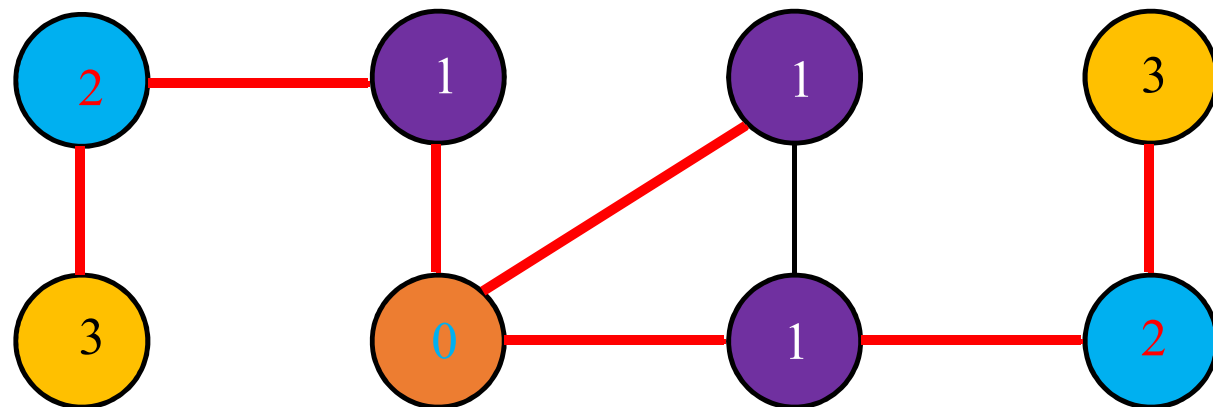
Review of Graph Search Algorithms

- “Explore” a graph, turning it into a tree
 - One vertex at a time
 - Expand *frontier* of explored vertices *across* the *breadth* of the frontier
- Builds a *tree* over the graph
 - Pick a *source vertex* to be the root
 - Find (“discover”) its *children*, then *their children*, etc.



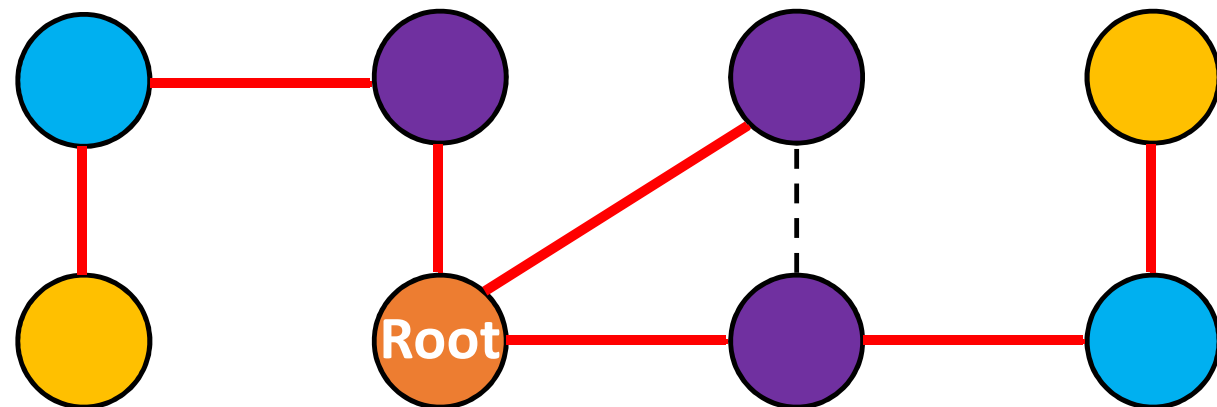
Review of Graph Search Algorithms

- “Explore” a graph, turning it into a tree
 - One vertex at a time
 - Expand *frontier* of explored vertices *across* the *breadth* of the frontier
- Builds a *tree* over the graph
 - Pick a *source vertex* to be the root
 - Find (“discover”) its *children*, then *their children*, etc.

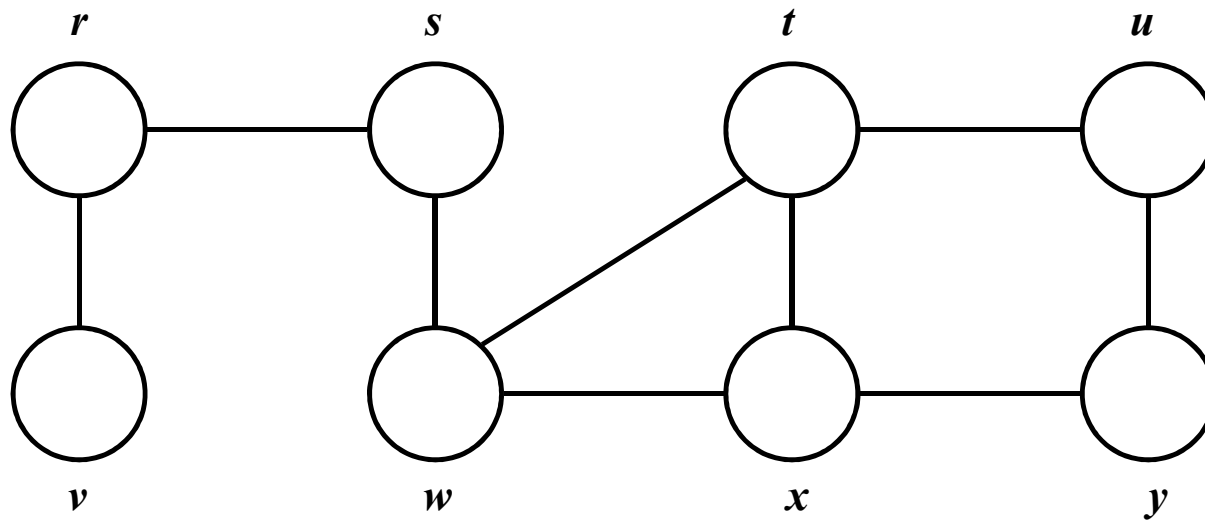


Review of Graph Search Algorithms

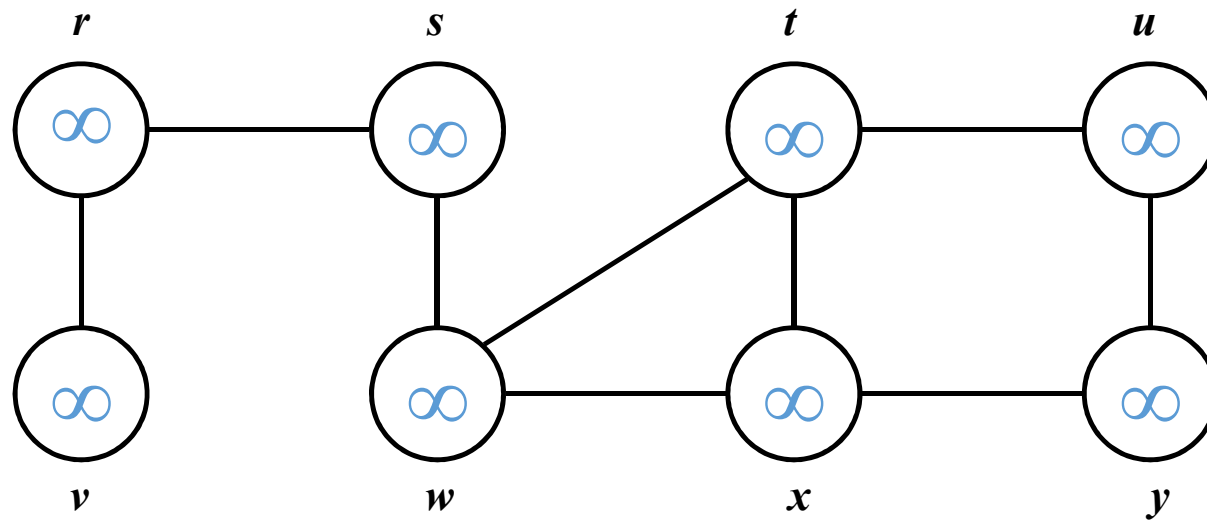
- “Explore” a graph, turning it into a tree
 - One vertex at a time
 - Expand **frontier** of explored vertices **across** the *breadth* of the frontier
- Builds a **tree** over the graph
 - Pick a *source vertex* to be the **root**
 - Find (“discover”) its **children**, then **their children**, etc.



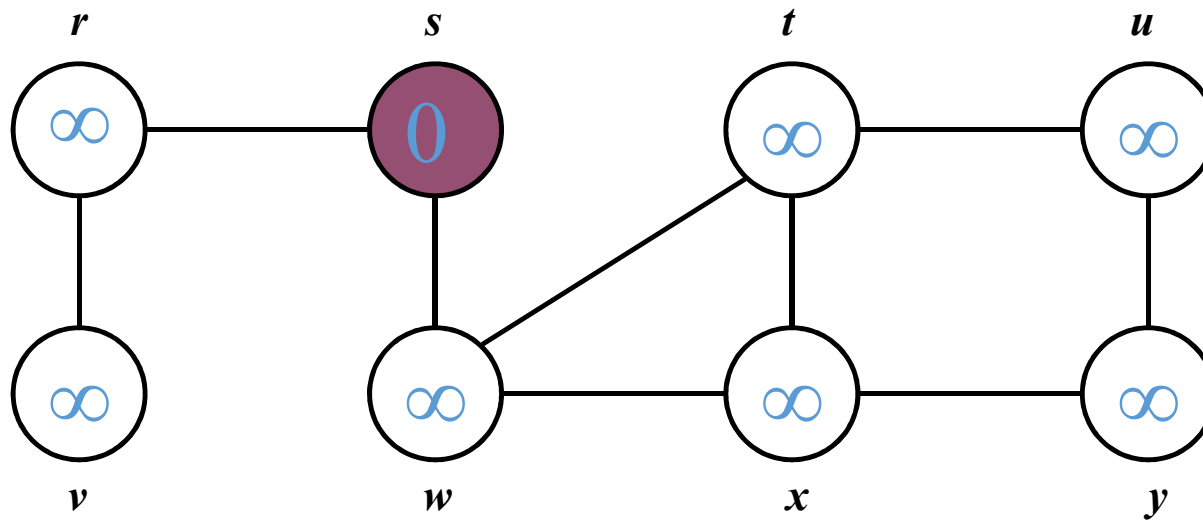
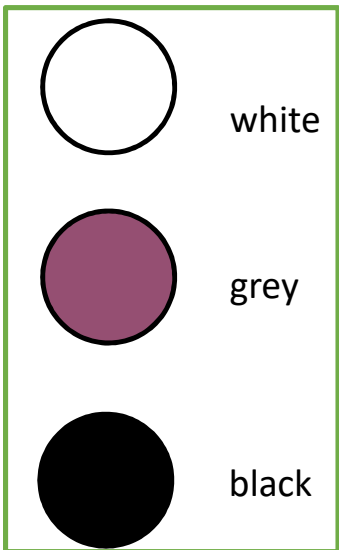
Review of Graph Search Algorithms: Example 2



Breadth-First Search: Example 2



Breadth-First Search: Example 2



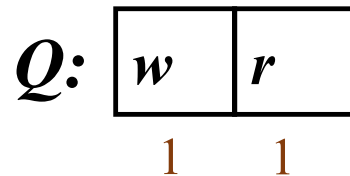
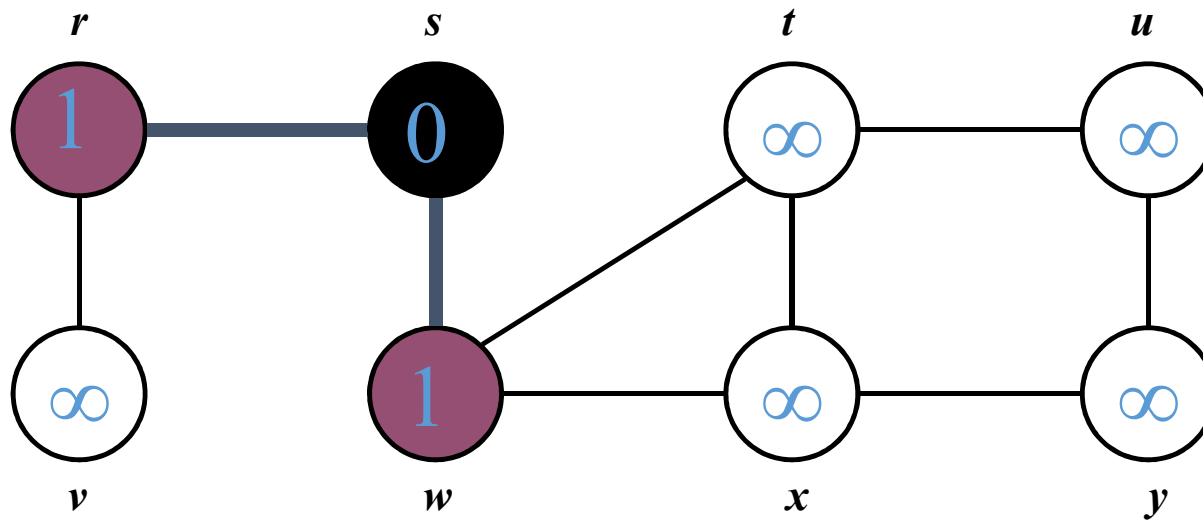
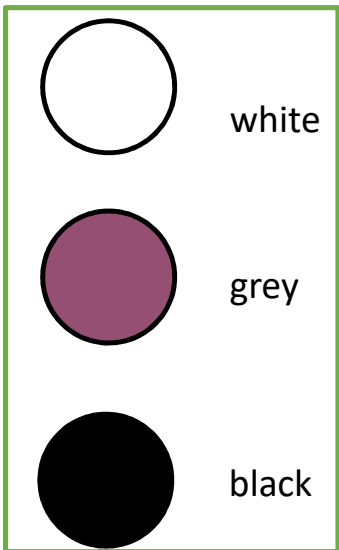
$Q:$ s
0

BFS(G, s)

```

1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4     $u.\pi = NIL$ 
5   $s.color = GRAY$ 
6   $s.d = 0$ 
7   $s.\pi = NIL$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11    $u = DEQUEUE(Q)$ 
12   for each  $v \in G.Adj[u]$ 
13     if  $v.color == WHITE$ 
14        $v.color = GRAY$ 
15        $v.d = u.d + 1$ 
16        $v.\pi = u$ 
17       ENQUEUE( $Q, v$ )
18    $u.color = BLACK$ 
    
```

Breadth-First Search: Example 2

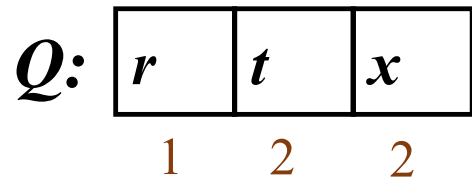
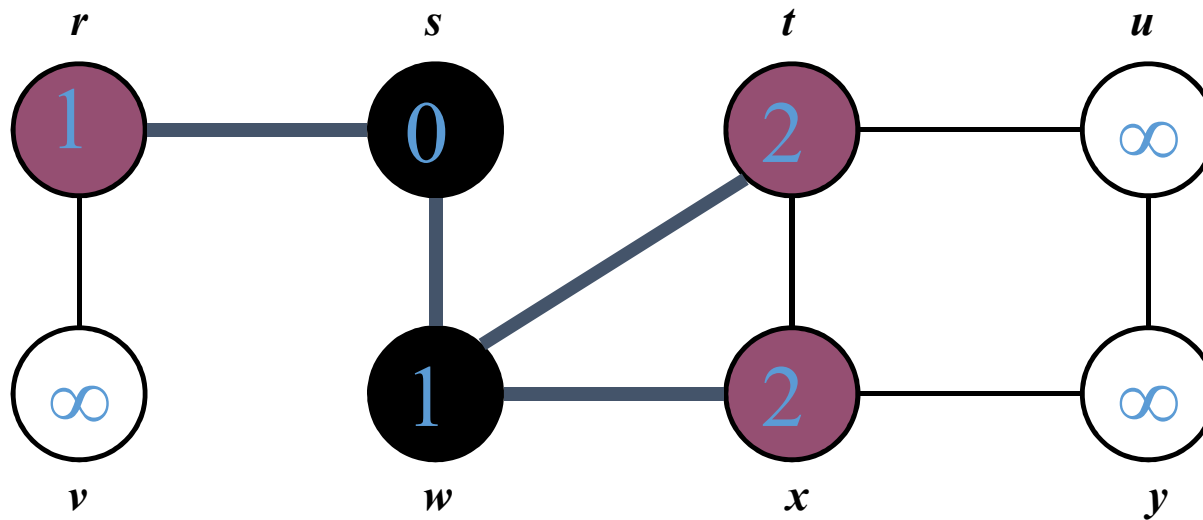
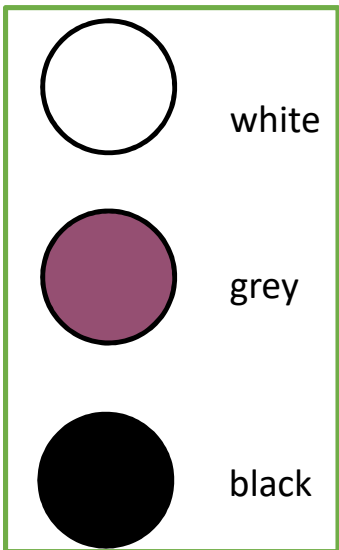


BFS(G, s)

```

1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = \text{WHITE}$ 
3     $u.d = \infty$ 
4     $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11    $u = \text{DEQUEUE}(Q)$ 
12   for each  $v \in G.Adj[u]$ 
13     if  $v.color == \text{WHITE}$ 
14        $v.color = \text{GRAY}$ 
15        $v.d = u.d + 1$ 
16        $v.\pi = u$ 
17       ENQUEUE( $Q, v$ )
18    $u.color = \text{BLACK}$ 
    
```

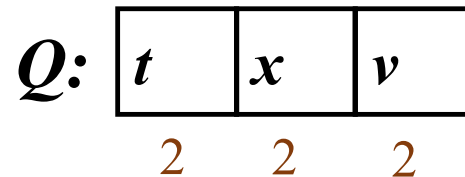
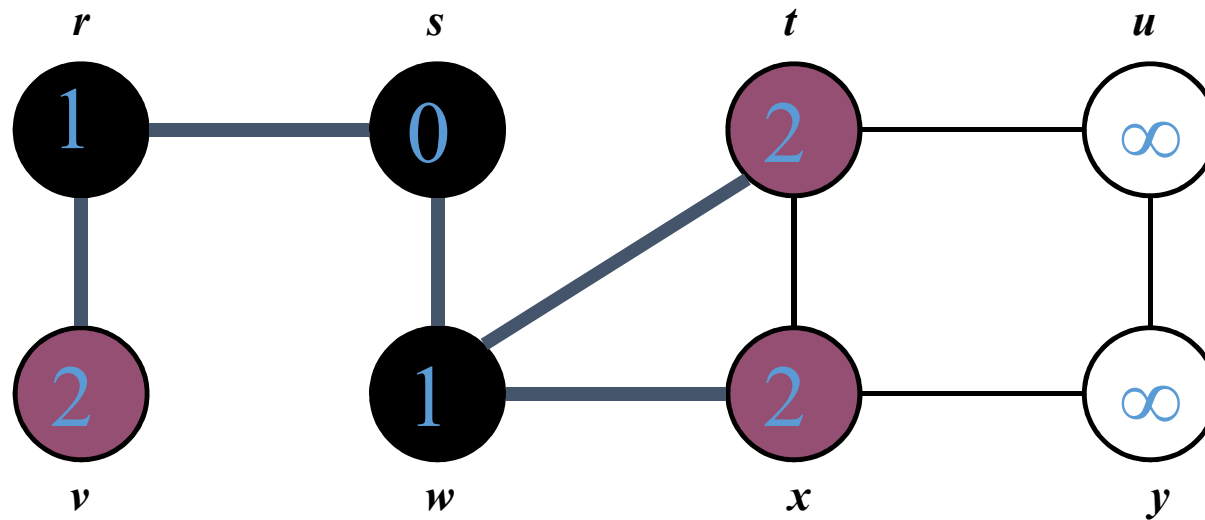
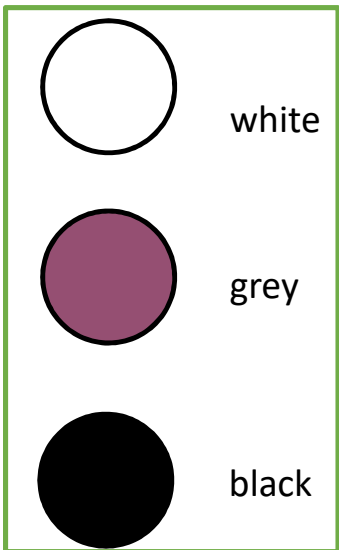
Breadth-First Search: Example 2



```

BFS(G, s)
1  for each vertex u ∈ G.V − {s}
2    u.color = WHITE
3    u.d = ∞
4    u.π = NIL
5  s.color = GRAY
6  s.d = 0
7  s.π = NIL
8  Q = ∅
9  ENQUEUE(Q, s)
10 while Q ≠ ∅
11   u = DEQUEUE(Q)
12   for each v ∈ G.Adj[u]
13     if v.color == WHITE
14       v.color = GRAY
15       v.d = u.d + 1
16       v.π = u
17       ENQUEUE(Q, v)
18   u.color = BLACK
    
```

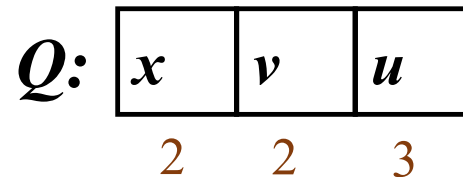
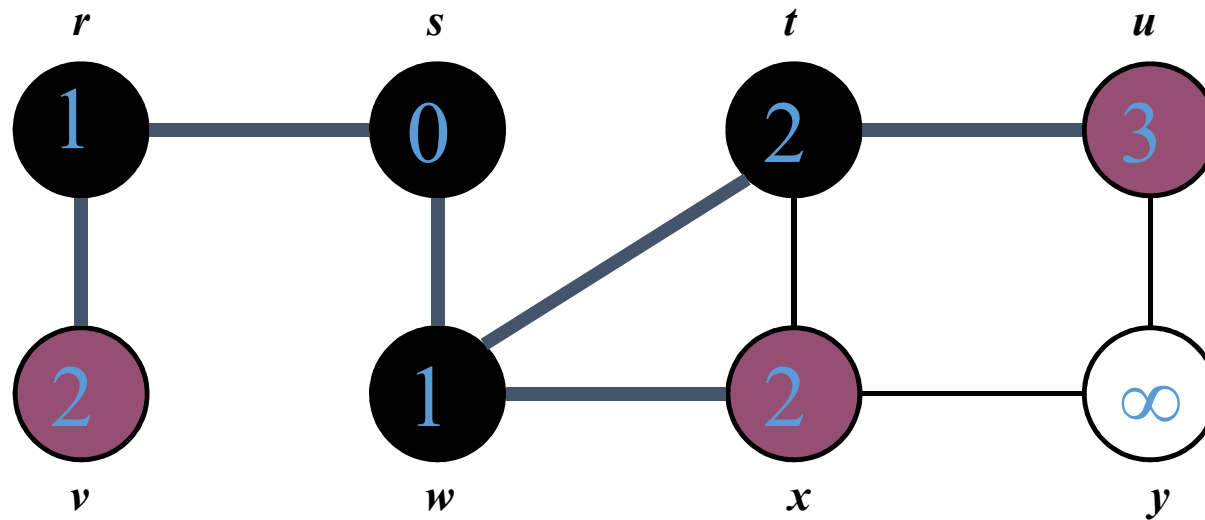
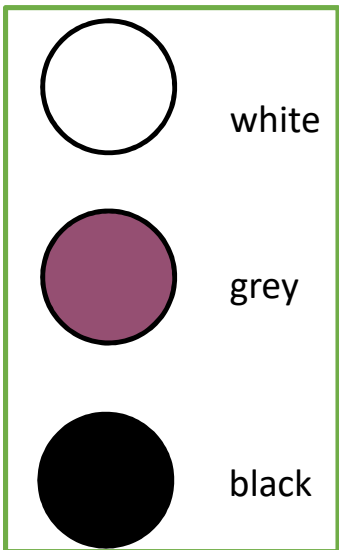
Breadth-First Search: Example 2



```

BFS(G, s)
1  for each vertex u ∈ G.V − {s}
2    u.color = WHITE
3    u.d = ∞
4    u.π = NIL
5  s.color = GRAY
6  s.d = 0
7  s.π = NIL
8  Q = ∅
9  ENQUEUE(Q, s)
10 while Q ≠ ∅
11   u = DEQUEUE(Q)
12   for each v ∈ G.Adj[u]
13     if v.color == WHITE
14       v.color = GRAY
15       v.d = u.d + 1
16       v.π = u
17       ENQUEUE(Q, v)
18   u.color = BLACK
    
```

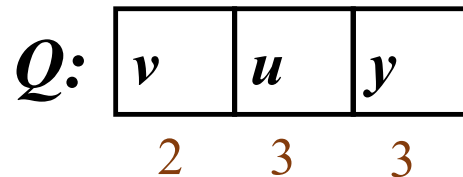
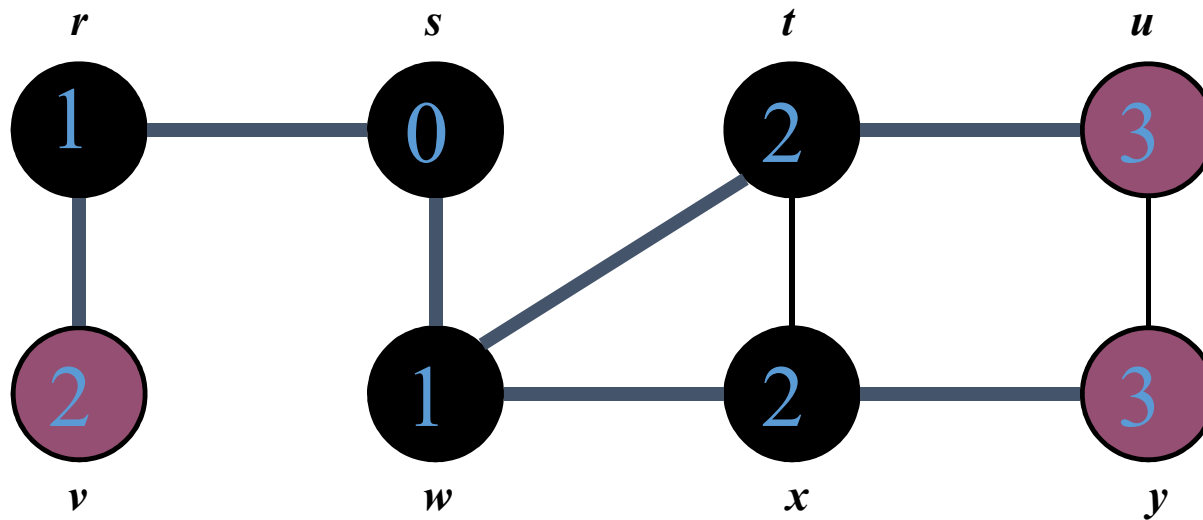
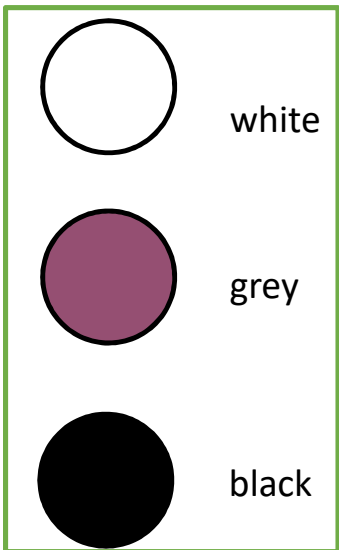
Breadth-First Search: Example 2



```

BFS( $G, s$ )
1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4     $u.\pi = NIL$ 
5   $s.color = GRAY$ 
6   $s.d = 0$ 
7   $s.\pi = NIL$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11    $u = DEQUEUE(Q)$ 
12   for each  $v \in G.Adj[u]$ 
13     if  $v.color == WHITE$ 
14        $v.color = GRAY$ 
15        $v.d = u.d + 1$ 
16        $v.\pi = u$ 
17       ENQUEUE( $Q, v$ )
18    $u.color = BLACK$ 
    
```

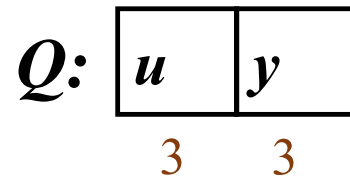
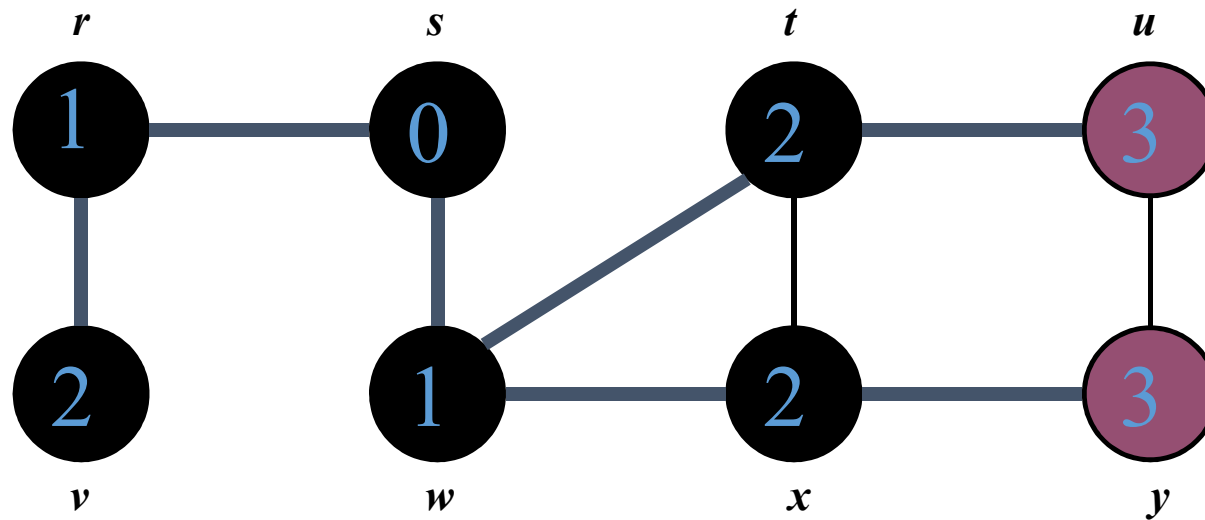
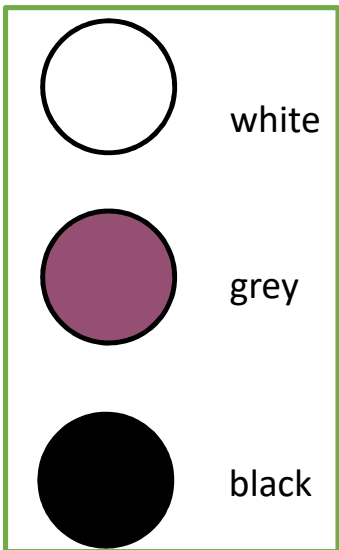
Breadth-First Search: Example 2



```

BFS( $G, s$ )
1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4     $u.\pi = NIL$ 
5   $s.color = GRAY$ 
6   $s.d = 0$ 
7   $s.\pi = NIL$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11    $u = DEQUEUE(Q)$ 
12   for each  $v \in G.Adj[u]$ 
13     if  $v.color == WHITE$ 
14        $v.color = GRAY$ 
15        $v.d = u.d + 1$ 
16        $v.\pi = u$ 
17       ENQUEUE( $Q, v$ )
18    $u.color = BLACK$ 
    
```

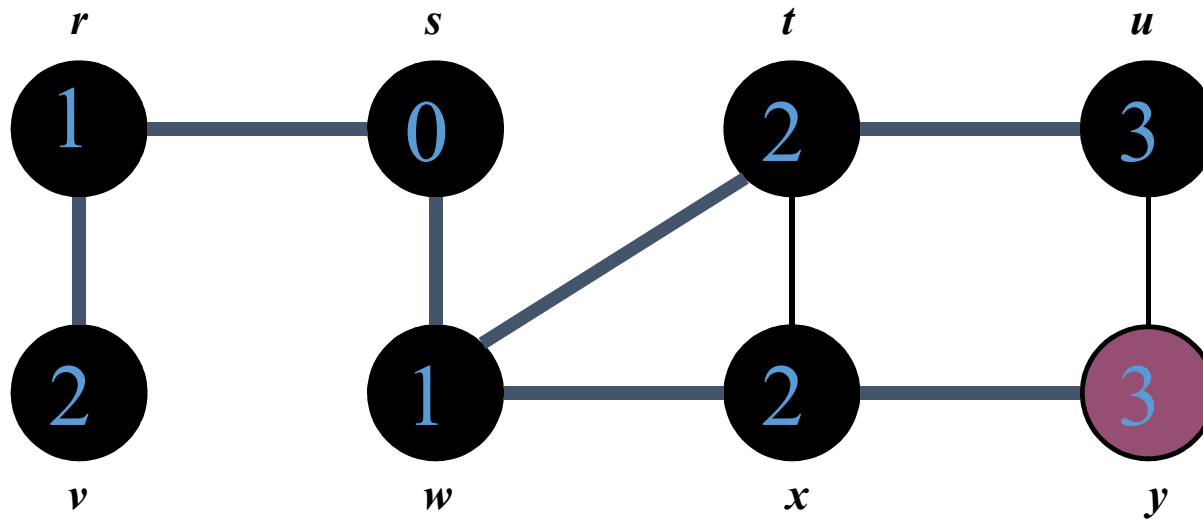
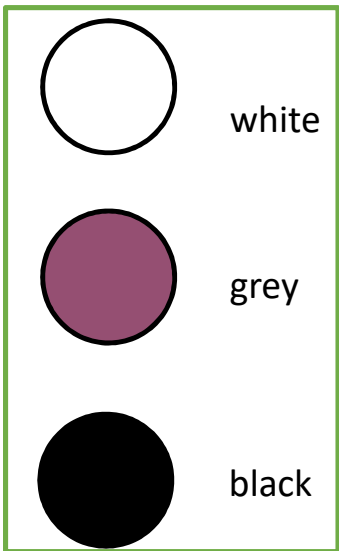
Breadth-First Search: Example 2



```

BFS( $G, s$ )
1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4     $u.\pi = NIL$ 
5   $s.color = GRAY$ 
6   $s.d = 0$ 
7   $s.\pi = NIL$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11    $u = DEQUEUE(Q)$ 
12   for each  $v \in G.Adj[u]$ 
13     if  $v.color == WHITE$ 
14        $v.color = GRAY$ 
15        $v.d = u.d + 1$ 
16        $v.\pi = u$ 
17       ENQUEUE( $Q, v$ )
18    $u.color = BLACK$ 
    
```

Breadth-First Search: Example 2

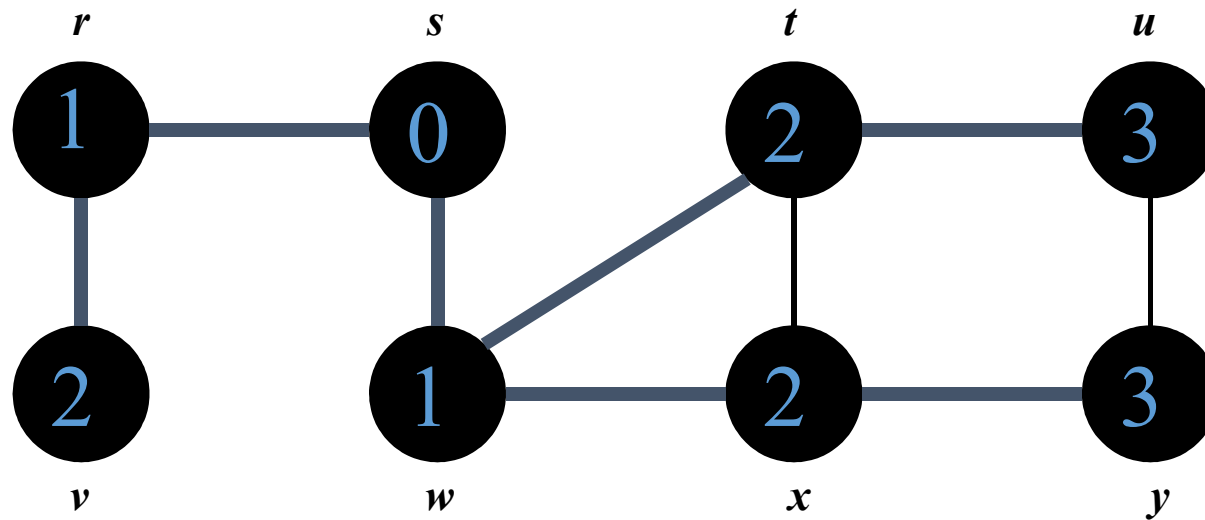
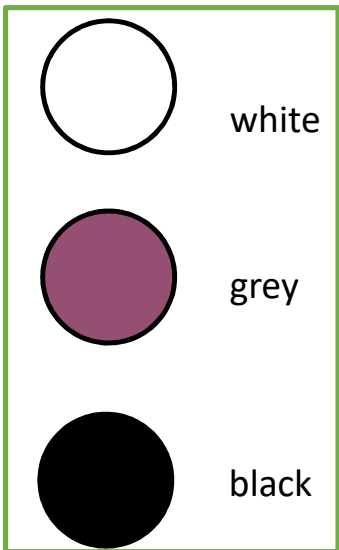


$Q:$ y
3

```

BFS( $G, s$ )
1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4     $u.\pi = NIL$ 
5   $s.color = GRAY$ 
6   $s.d = 0$ 
7   $s.\pi = NIL$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11    $u = DEQUEUE(Q)$ 
12   for each  $v \in G.Adj[u]$ 
13     if  $v.color == WHITE$ 
14        $v.color = GRAY$ 
15        $v.d = u.d + 1$ 
16        $v.\pi = u$ 
17       ENQUEUE( $Q, v$ )
18    $u.color = BLACK$ 
    
```

Breadth-First Search: Example 2

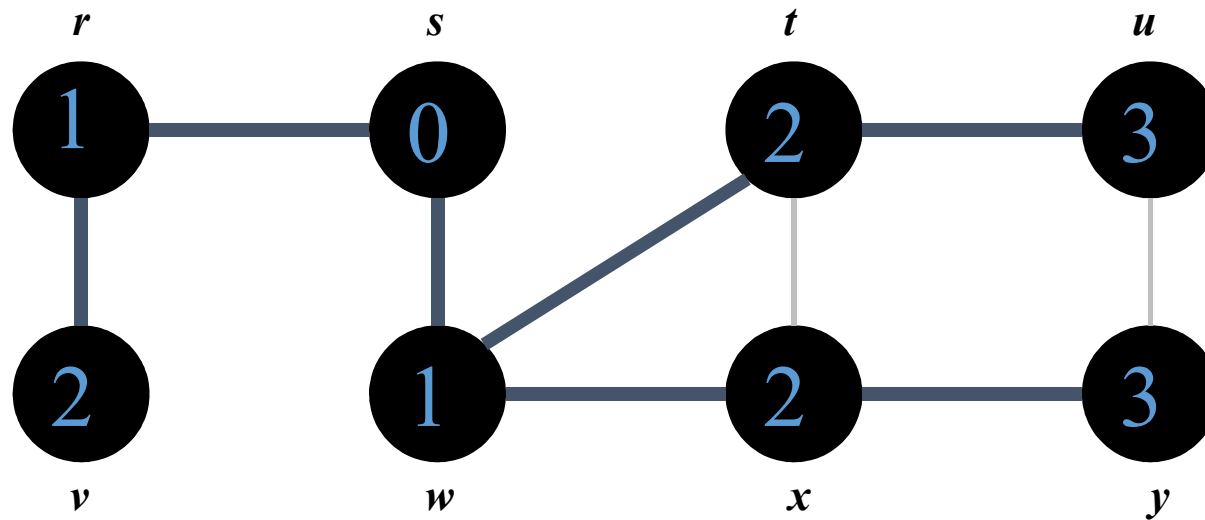


$Q: \emptyset$

```

BFS( $G, s$ )
1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4     $u.\pi = NIL$ 
5   $s.color = GRAY$ 
6   $s.d = 0$ 
7   $s.\pi = NIL$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11    $u = DEQUEUE(Q)$ 
12   for each  $v \in G.Adj[u]$ 
13     if  $v.color == WHITE$ 
14        $v.color = GRAY$ 
15        $v.d = u.d + 1$ 
16        $v.\pi = u$ 
17       ENQUEUE( $Q, v$ )
18    $u.color = BLACK$ 
    
```

Breadth-First Search: Example 2

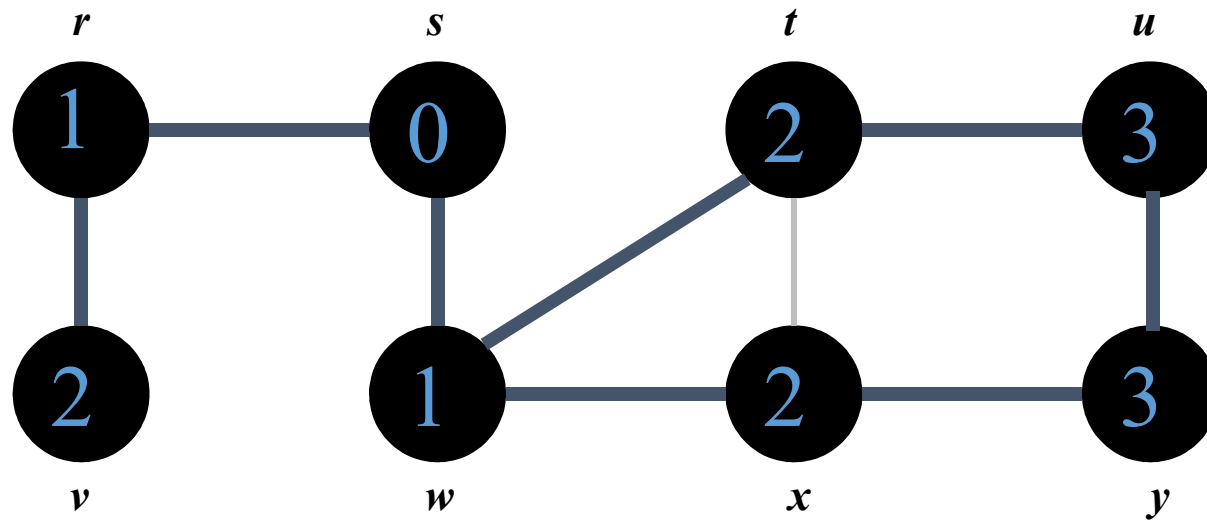


Removing the UNUSED (faded) edges,

Remaining edges span all vertices: **Spanning tree**

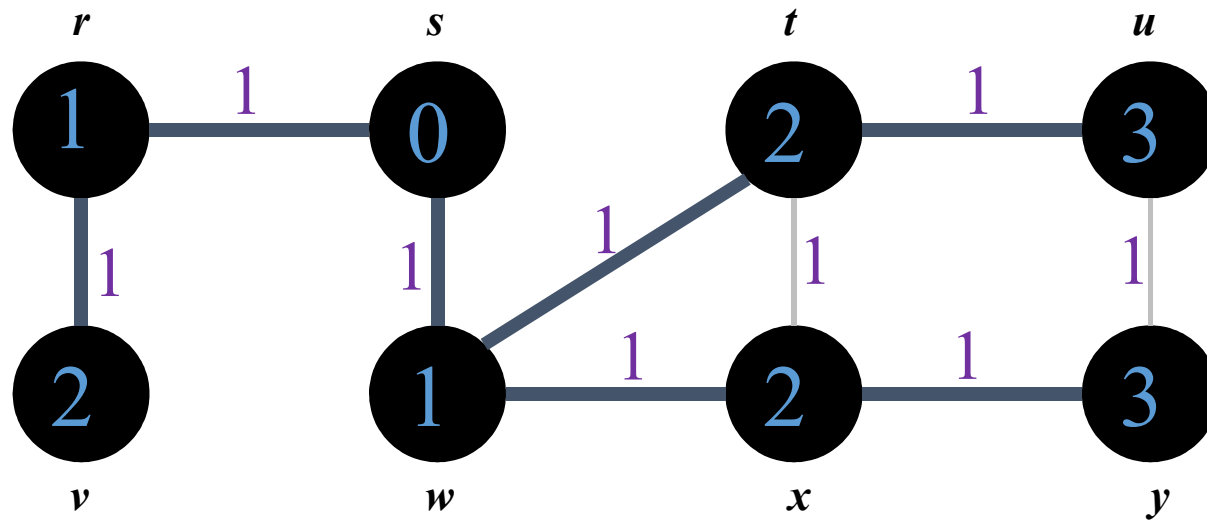
The values represent the minimum distances from source: **single source shortest path**

Breadth-First Search: Example 2



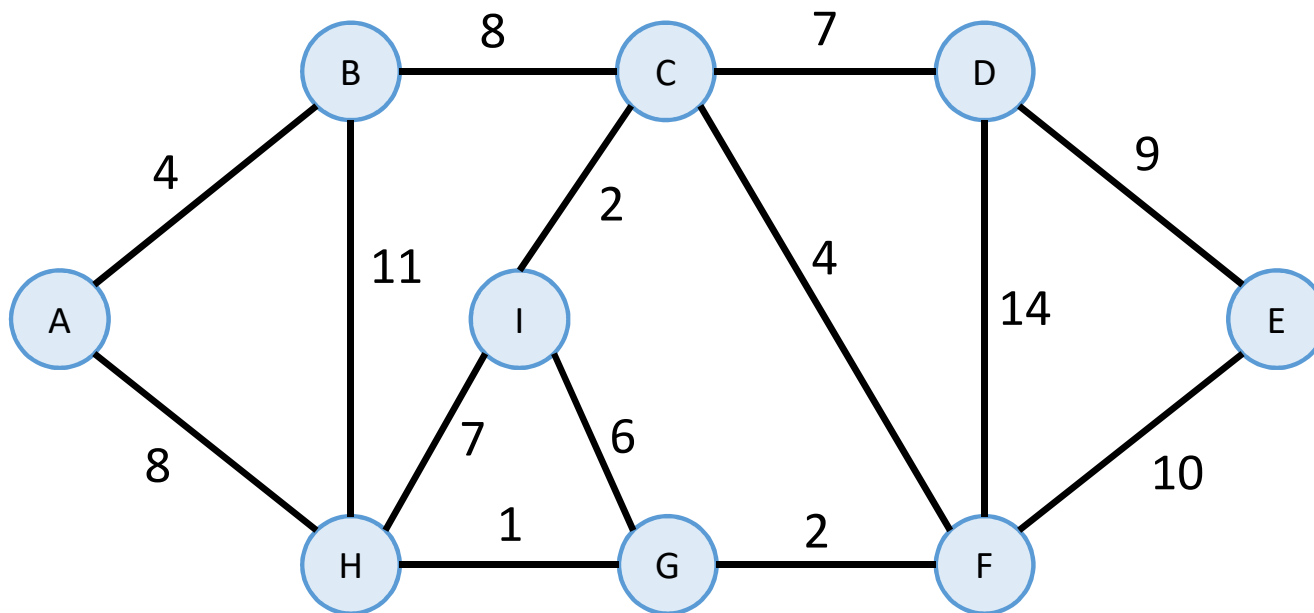
Adding edge (u, y) will make a cycle

Breadth-First Search: Example 2



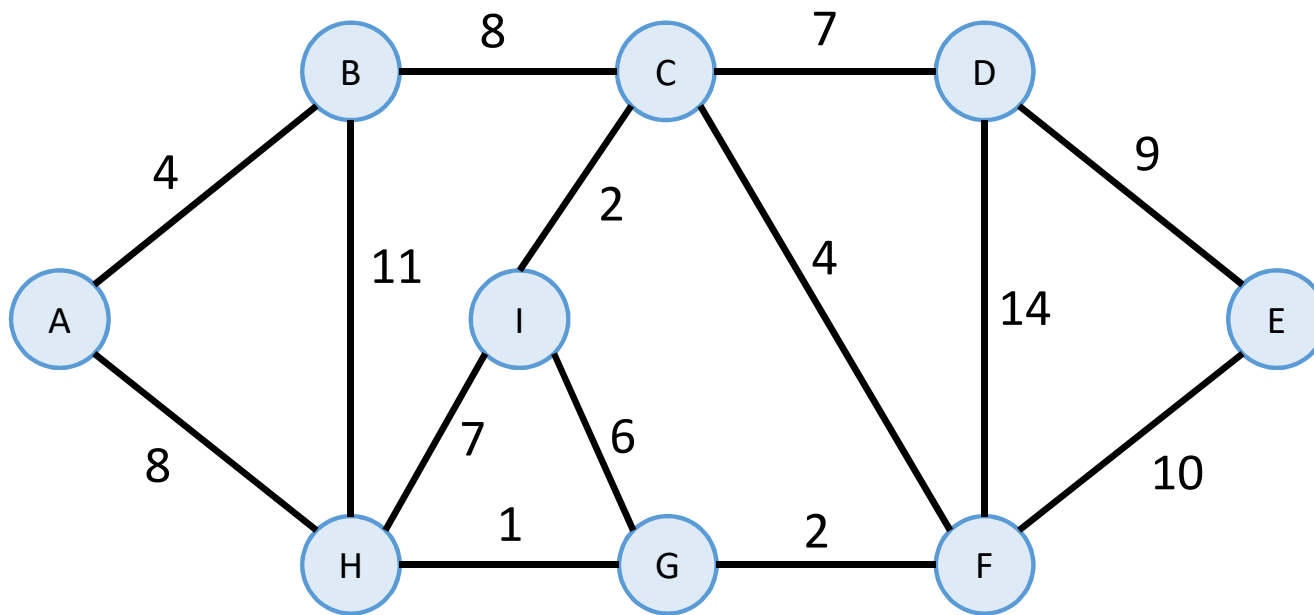
We assume all edges having same weights, e.g., 1

Another Example



A graph with different edge-weights

Applications

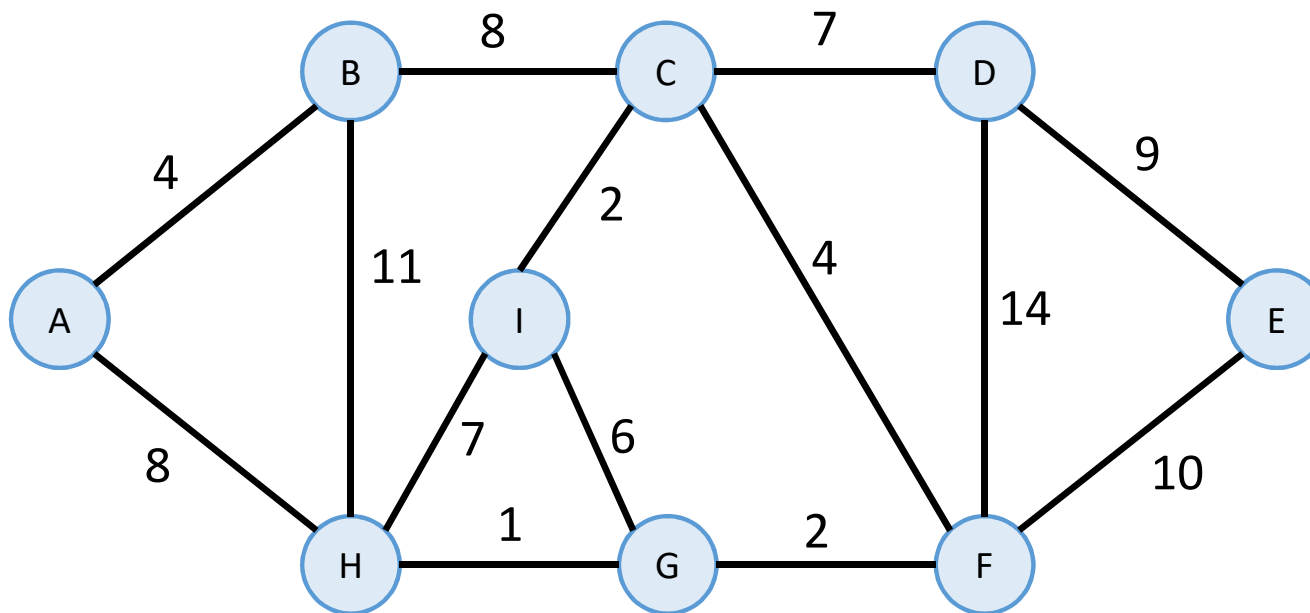


Assume: vertices are cities and edge-weights are distances

Problem: find shortest distances from city A

Applications

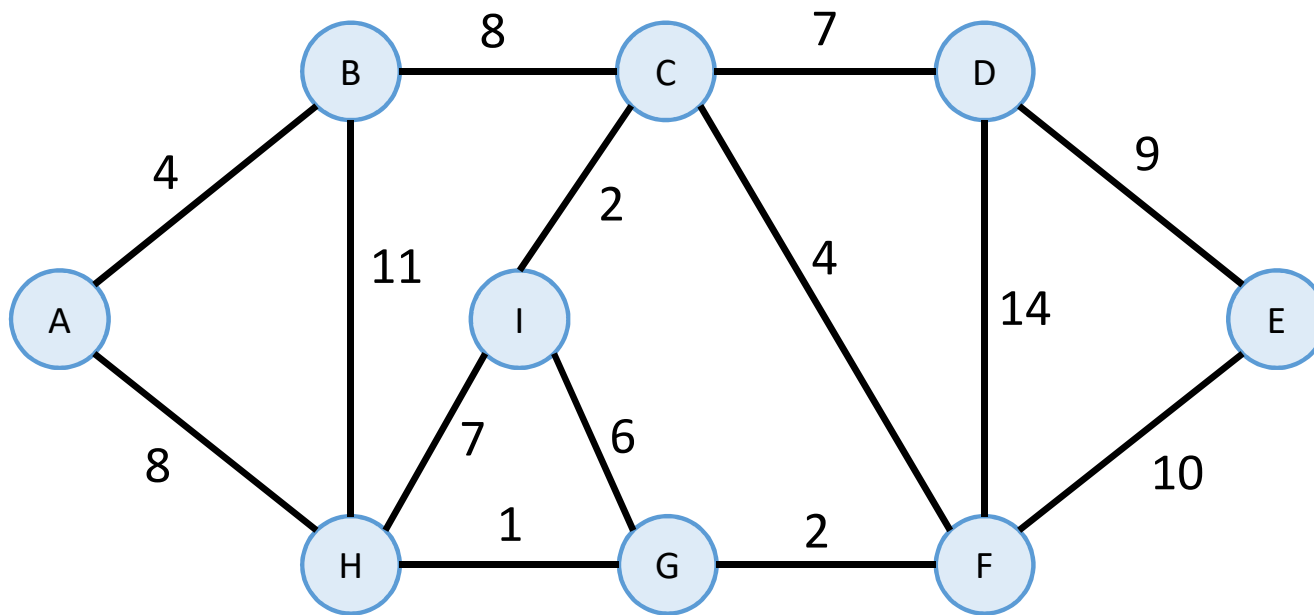
Single source
shortest path
(SSSP) problem



Assume: vertices are cities and edge-weights are distances

Problem: find shortest distances from city A

Applications

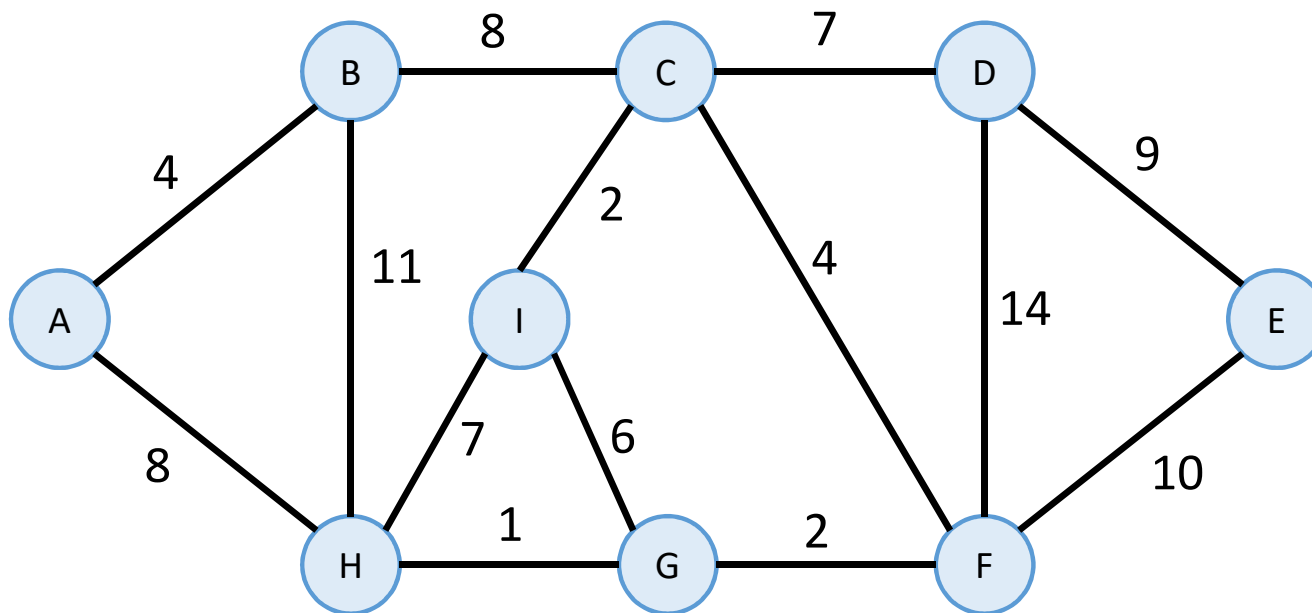


Assume: vertices are cities and edge-weights are distances

Problem: find minimum cable length for electricity supply

Applications

Minimum spanning
tree (MST) problem



Assume: vertices are cities and edge-weights are distances

Problem: find minimum cable length for electricity supply

Applications



Electronic circuit board:

Wiring Problem: interconnect n points with $n-1$ wires with minimum length

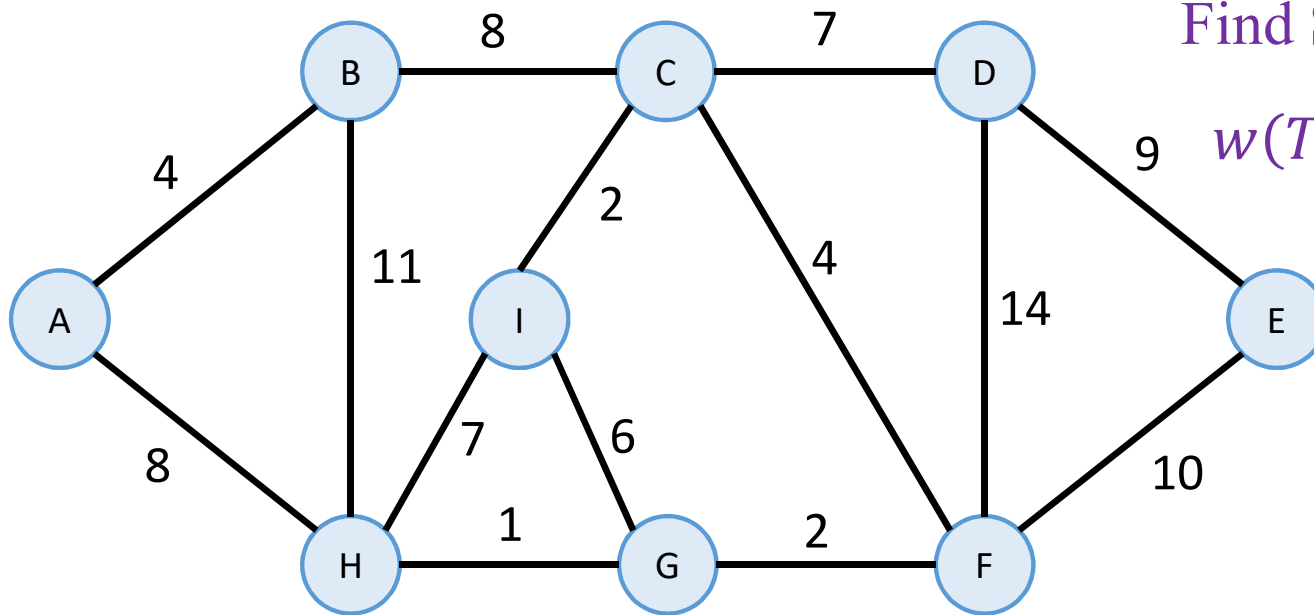
Applications



Electronic circuit board:

Wiring Problem: interconnect n points with $n-1$ wires with minimum length

Solution of Wiring Problem



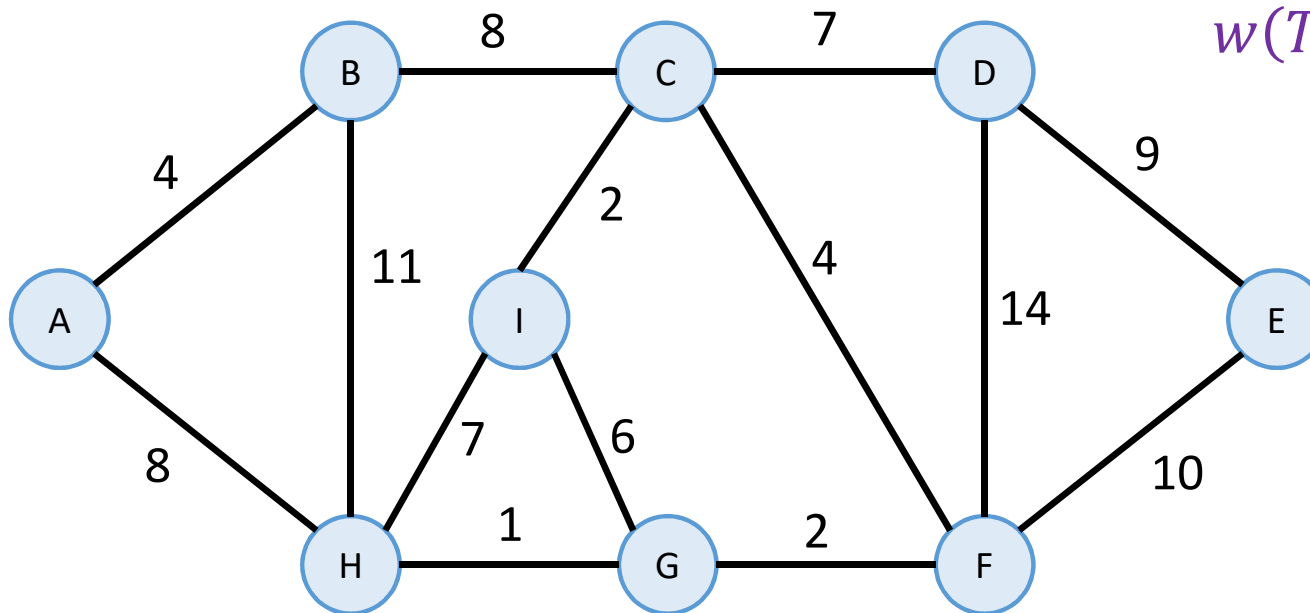
Modelling:

Graph, $G=(V, E)$ is
connected and weighted
Find Subset $T \subseteq E$ so that

$$w(T) = \sum_{(u,v) \in T} w(u, v)$$

is minimized

Solution of Wiring Problem



Modelling:

Graph, $G=(V, E)$

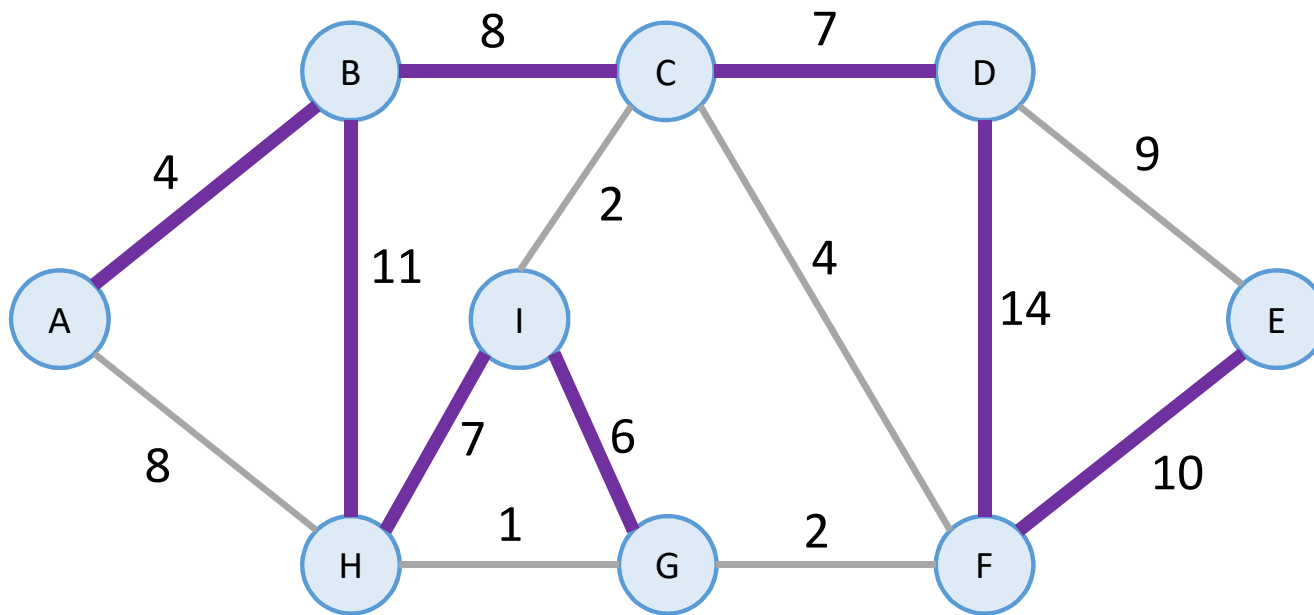
Find Subset $T \subseteq E$ so that

$$w(T) = \sum_{(u,v) \in T} w(u, v)$$

is minimized

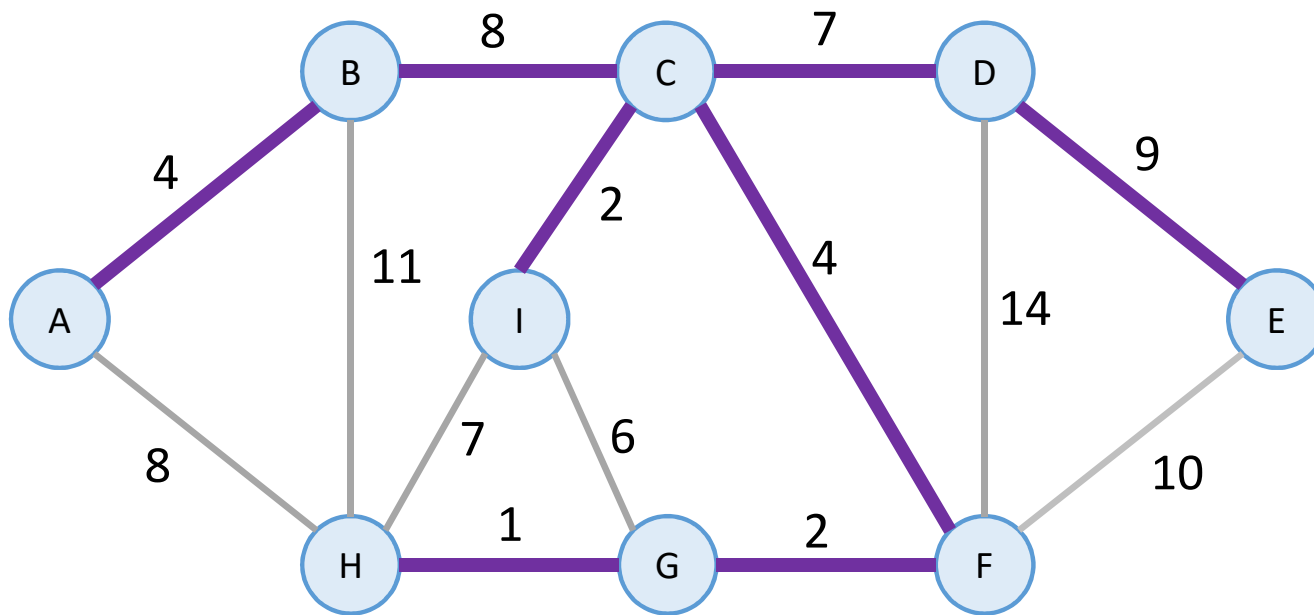
Subset T must reach each vertex exactly once: minimum spanning tree

Spanning Tree



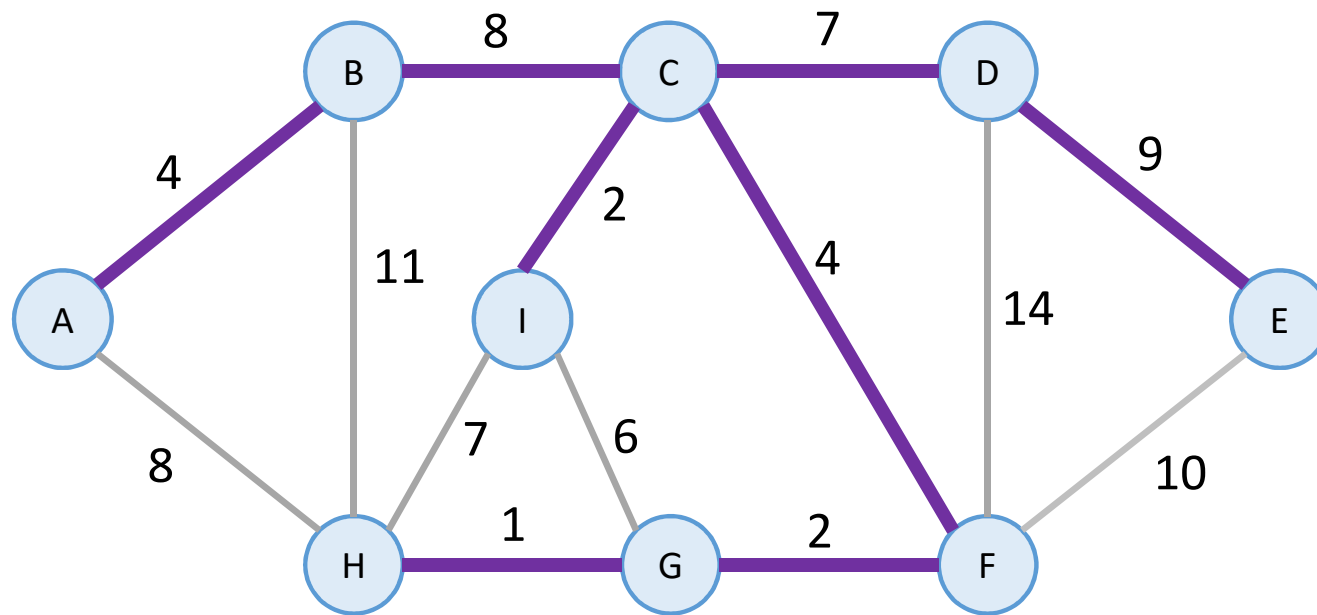
Spanning tree with cost 67

Spanning Tree



Spanning tree with cost 37

Minimum Spanning Tree



Minimum Spanning tree with cost 37

Finding Minimum Spanning Tree

Given a graph, $G=(V, E)$, connected and undirected

With weight function, $w: E \rightarrow R$

Find an MST with the following greedy approach

GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

Finding Minimum Spanning Tree

GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

A is a subset of some MST

Grows gradually

One **safe edge** is added in each iteration

After termination, A is an MST

Finding Minimum Spanning Tree

GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

The *loop invariant* is

Prior to each iteration, A is a subset of some minimum spanning tree.

Finding Minimum Spanning Tree

GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

The *loop invariant*:

Prior to each iteration, A is a subset of some minimum spanning tree.

Initialization: After line 1, the set A trivially satisfies the loop invariant
because, A is empty

Maintenance:

Termination:

Finding Minimum Spanning Tree

GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

The *loop invariant*:

Prior to each iteration, A is a subset of some minimum spanning tree.

Maintenance: The loop in lines 2–4 maintains the invariant by adding only safe edges.

Termination: All edges added to A are in a minimum spanning tree, and so the set A after termination must be an MST

Finding Minimum Spanning Tree

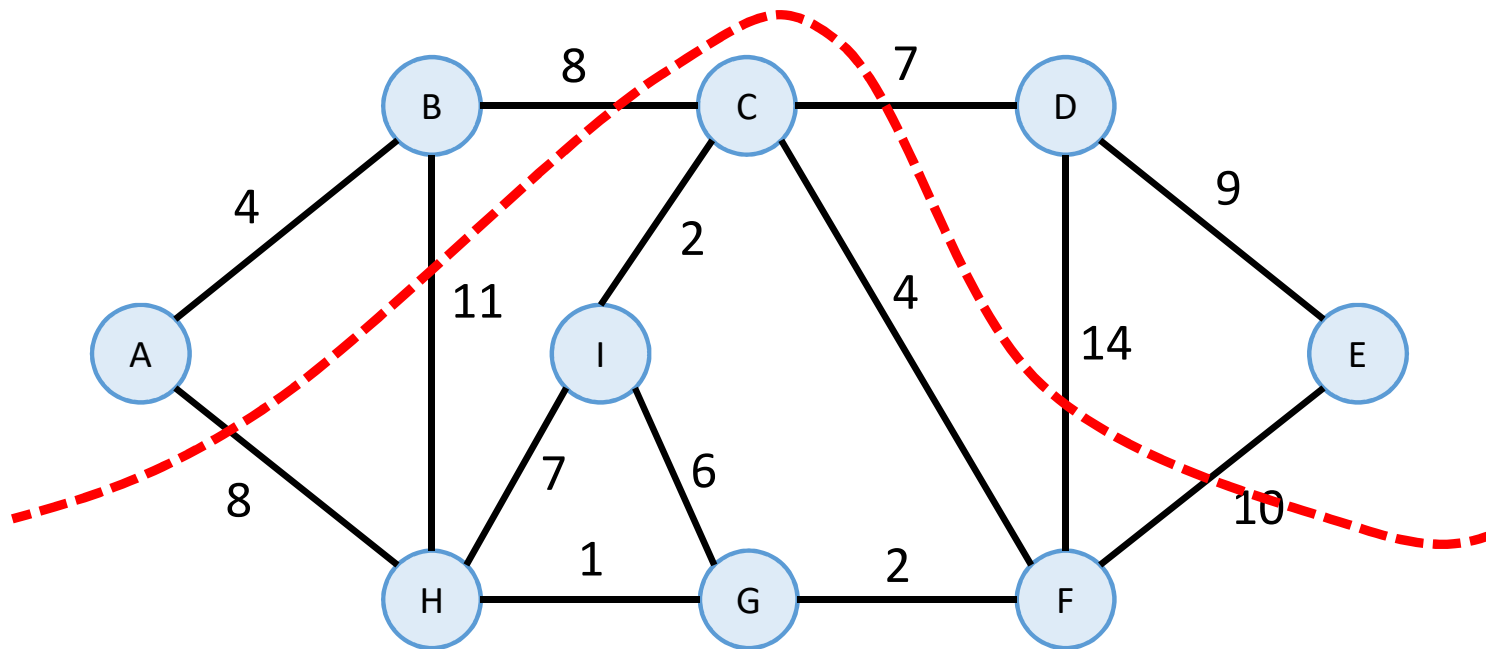
GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

We find a safe edge (u, v) so that $A \cup \{(u, v)\}$ is also subset of some MST.

Finding a safe edge for a growing MST

Cut in graph $G=(V, E)$: a partition of V : $(S, V-S)$

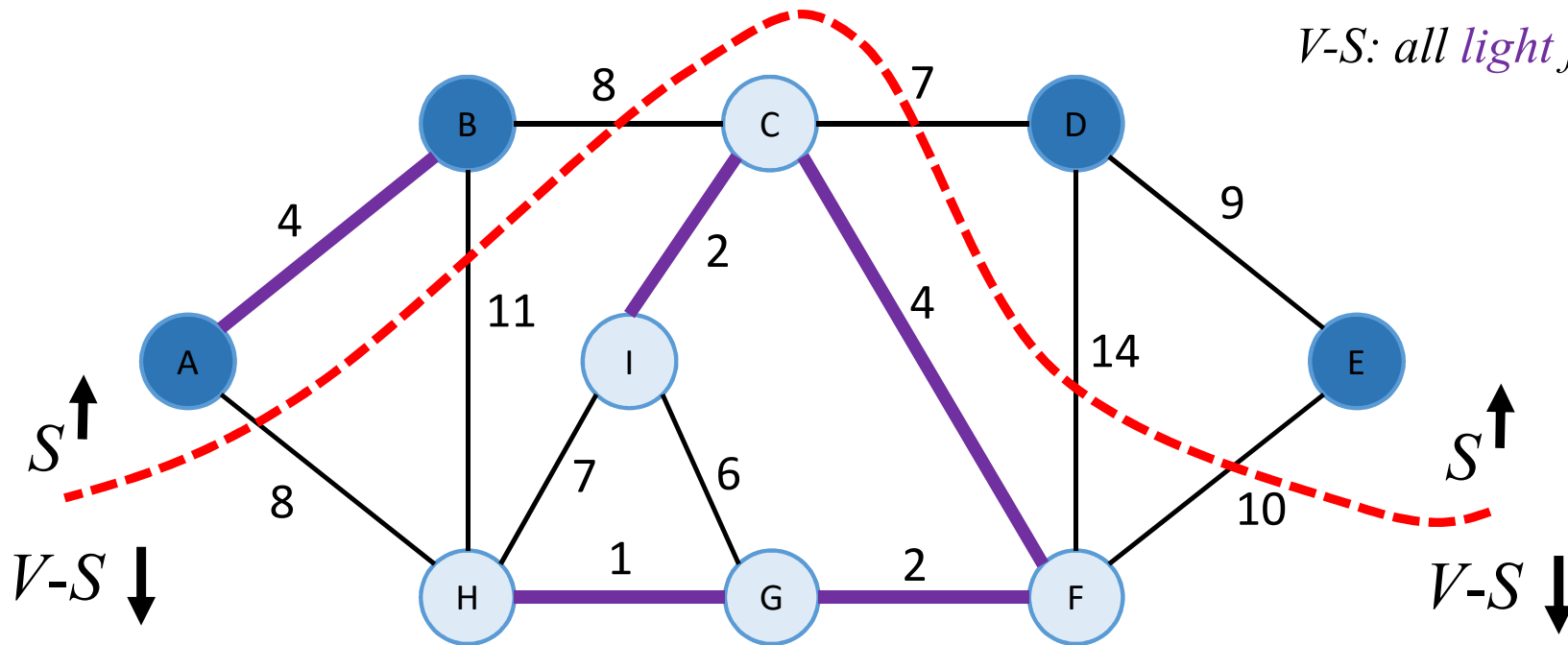


Finding a safe edge for a growing MST

Cut in graph $G=(V, E)$: a partition of V : $(S, V-S)$

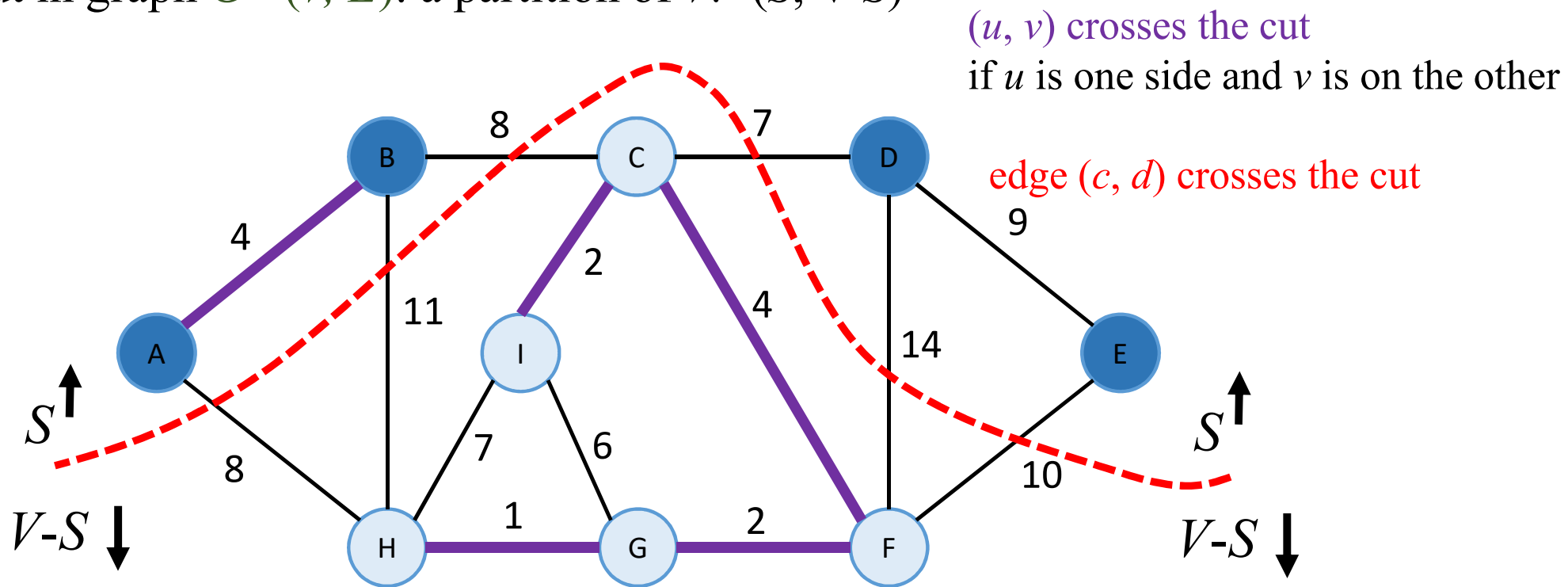
S : all *dark* filled vertices

$V-S$: all *light* filled vertices



Finding a safe edge for a growing MST

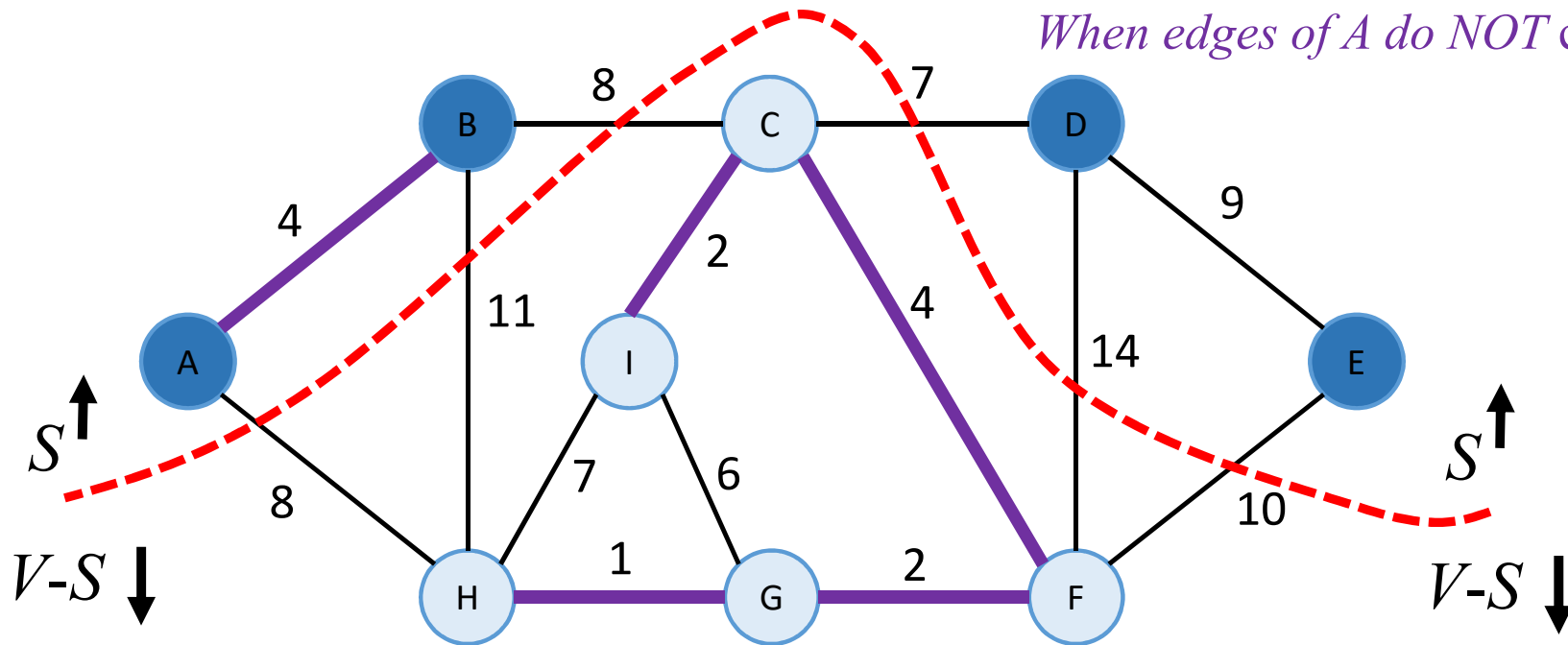
Cut in graph $G=(V, E)$: a partition of V : $(S, V-S)$



Finding a safe edge for a growing MST

Let A is the set of thick edges

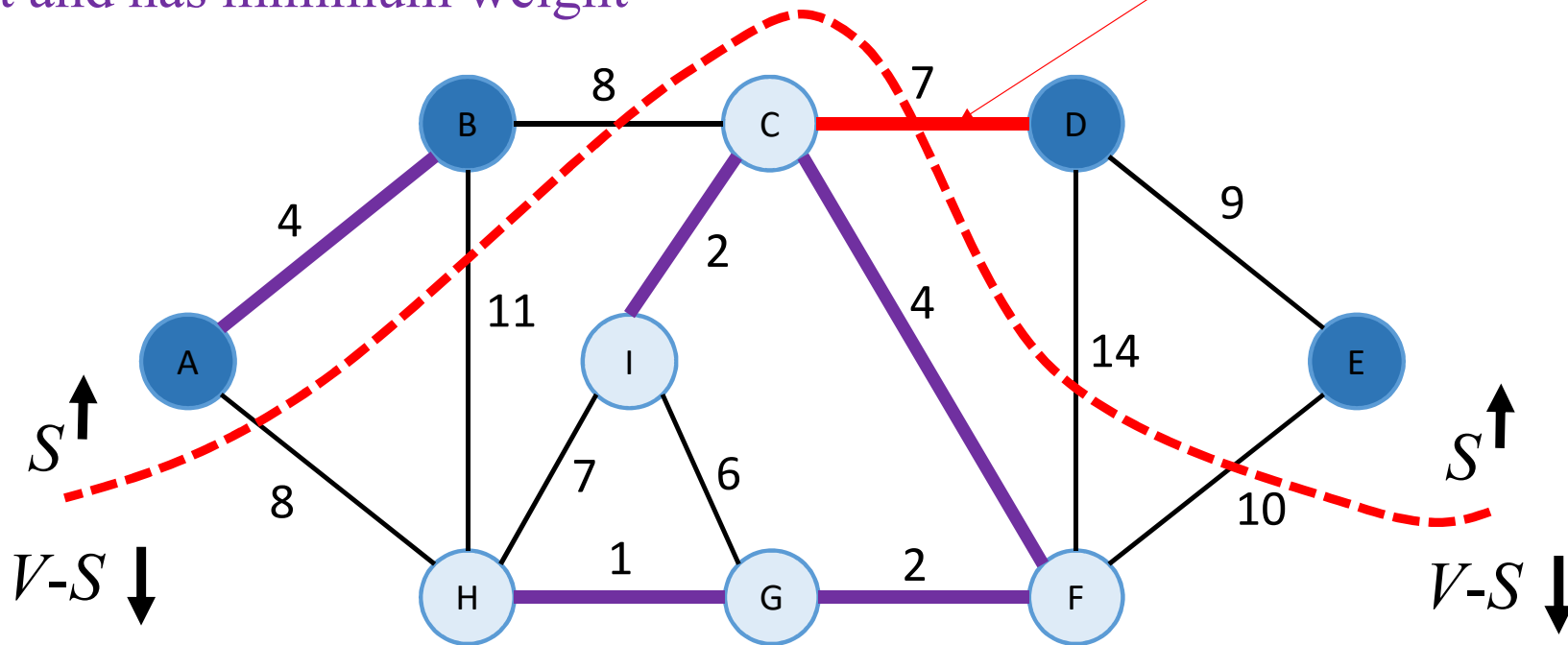
The cut $(S, V-S)$ respects A
When edges of A do NOT cross the cut



Finding a safe edge for a growing MST

Light edge (u, v) crosses the cut and has minimum weight

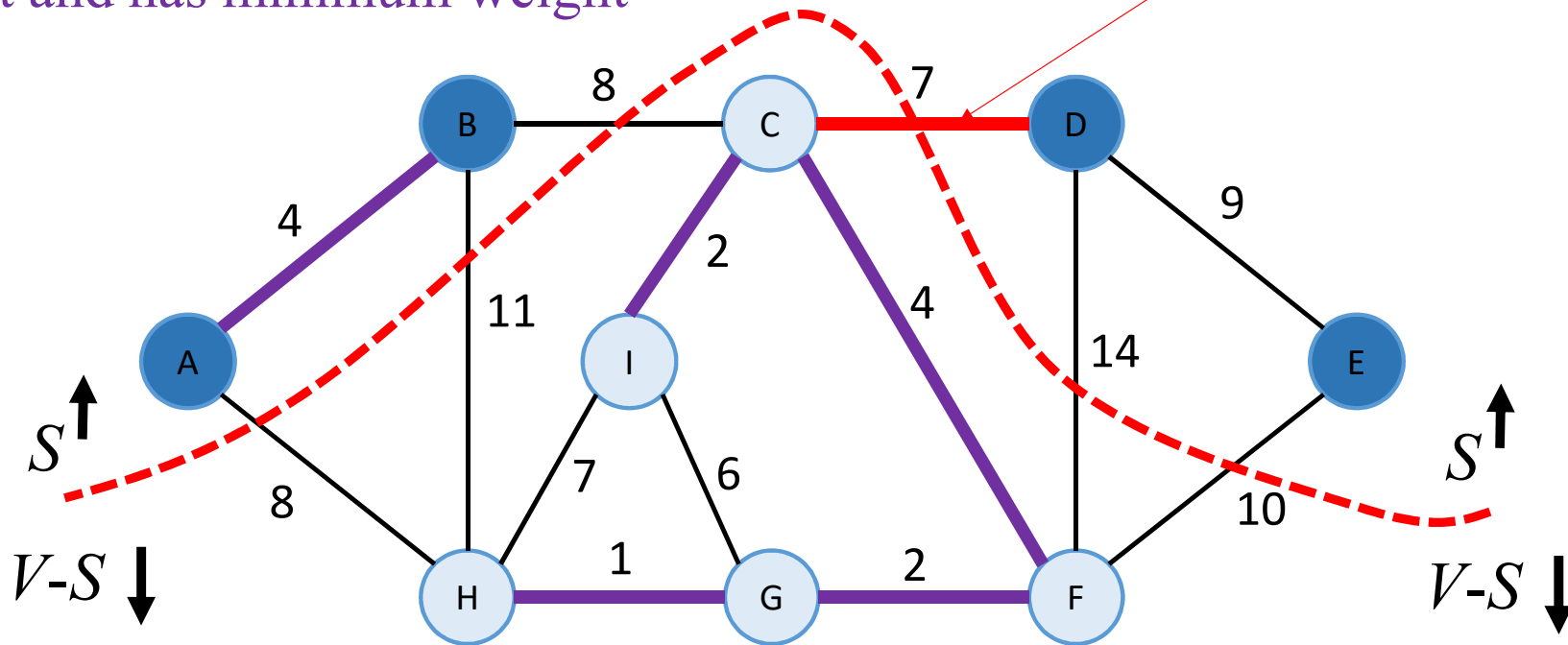
The edge (c, d) is a light edge



Finding a safe edge for a growing MST

Light edge (u, v) crosses the cut and has minimum weight

The edge (c, d) is a light edge



Finding a safe edge for a growing MST

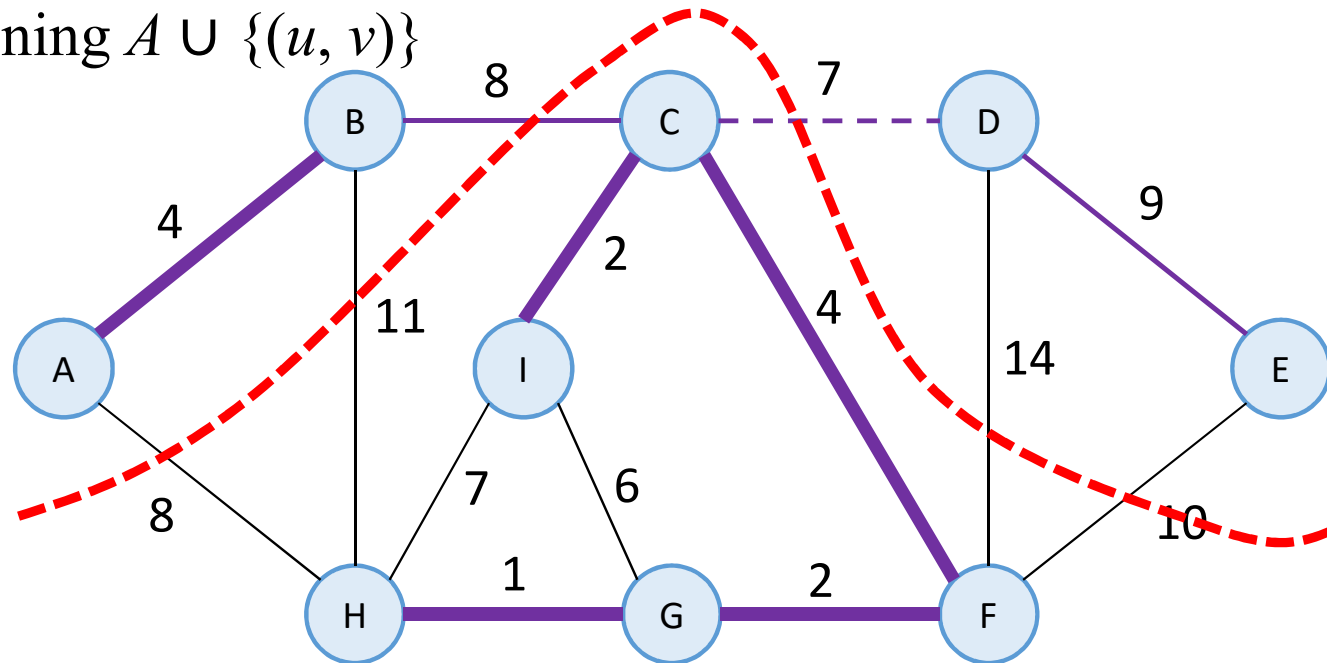
Theorem 23.1:

Let A be a set of edges, and consider a cut that respects A .

there is an MST containing A .

(u, v) be a light edge.

Then there is an MST containing $A \cup \{(u, v)\}$



Finding a safe edge for a growing MST

Theorem 23.1:

Let A be a set of edges, and consider a cut that respects A .

there is an MST containing A .

(u, v) be a light edge.

Then there is an MST containing $A \cup \{(u, v)\}$

Explanation:

A = set of **thick** edges

T = an MST: all violet edges

$A \subset T$

(c, d) is a light edge.

Then, $A \cup \{(c, d)\} \subset T$

